Recitation 11

Multi-file programs and linked lists

Reichman University
Efi Arazi School of Computer Science

Recitation overview

- **How to organize a software project?**
- The linked list code module – explore and modify
- Review file structure for HW #6

Organization of a software project

- Non-trivial problems require large code bases.
- Often multiple people will be involved in a project.
- How to prevent programmers from interfering with each other's work?
- How to effectively communicate between programmers about code that's not yet written or is subject to frequent change?
- Even if you're alone, you can only keep so much in your head at once.

Organization of a software project

- We separate the program into sub-components (modules, classes).
- Each component has a well-defined, well-documented, **minimalistic**, public interface, that changes rarely.
- We, as users of that interface, don't care about the implementation – if or when it changes.

Organization of a software project

- In C, headers (.h) are public interfaces
- Source code files (.c) are the implementation behind those interfaces

Multi-file programs – recap

Code modules:

- A module in C codes for a certain functionality (similar to a class)
- Modules typically contain:
  - Function definitions
  - Type definitions (struct types and/or object types)
  - Other items, e.g., symbolic constants, global variables, etc.
- Modules do not include a `main` function
  (so that you could use them in any program)
- An executable is generated by **linking** several modules together with an additional code file containing a main function.

Header files

The header file of a code module (.h) declares the components we wish to make public so that others can use

- ✓Functions (declarations)
- ✓Data types (typedef/struct commands)
- ✓Symbolic constants (#define directives)
- ✓Global variables (declarations)

Discussed in detail in class

Including a header file for module A "imports" their content, which allows other programs/modules to use the functions and data types of module A.

Code files

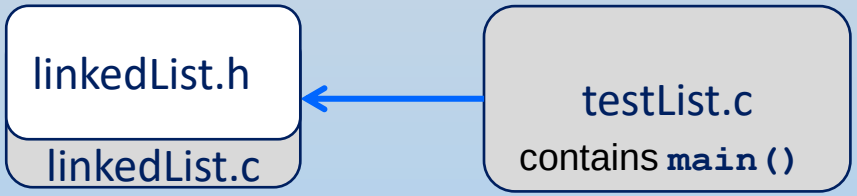
The source file of a code module (.c) contains an implementation of everything committed to in header file

- ✓Public functions (definition)
- ✓Private functions (definition)
- ✓Private data types (e.g., struct types)
- ✓Other possible private components (symbolic constants, types, etc.)

Multi-file programs – example 3

Program that uses a linked list:

- The source code implementing the linked list is given in linkedList.c
- The public components (functions / data types) are declare in linkedList.h
- Public data types and function of the linked list module are used in the program source file testList.c



Code in `/share/recitations/recitaion11/`

Recitation overview

- How to organize a software project?
- **The linked list code module – explore and modify**
- Review file structure for HW #6

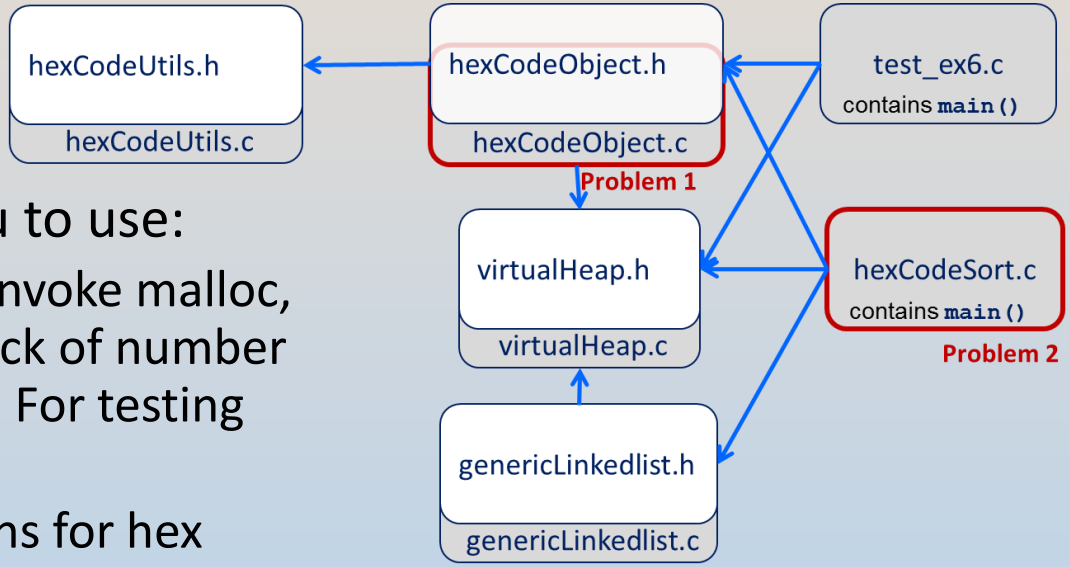
The linked list code module – explore and modify

- Demo on the console . . .

Recitation overview

- How to organize a software project?
- The linked list code module – explore and modify
- **Review file structure for HW #6**

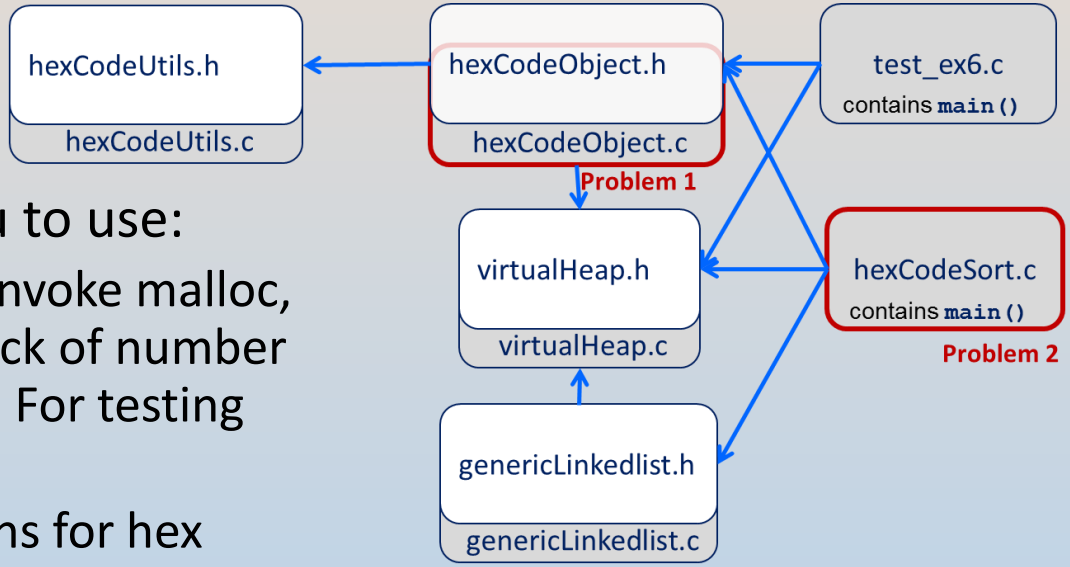
HW #6 code organization



- We provide 3 modules for you to use:
  - virtualHeap – functions that invoke malloc, realloc and free, and keep track of number of allocated blocks and bytes. For testing your memory allocation.
  - hexCodeUtils – utility functions for hex codes (some of which you implemented in HW #5).
  - genericLinkedList – an implementation of a generic linked list (can hold arbitrary objects)
- For each of these modules, we provide a header file and an object file. The object file is a pre-compiled version of the module

In Problem 1, we ask you to implement a module for a HexCode object. We provide the header for this module. The test file (test_ex6.c) contains a main function that calls these functions to test them.

HW #6 code organization



- We provide 3 modules for you to use:
  - virtualHeap – functions that invoke malloc, realloc and free, and keep track of number of allocated blocks and bytes. For testing your memory allocation.
  - hexCodeUtils – utility functions for hex codes (some of which you implemented in HW #5).
  - genericLinkedList – an implementation of a generic linked list (can hold arbitrary objects)
- For each of these modules, we provide a header file and an object file. The object file is a pre-compiled version of the module

- In Problem 2, we ask you to use this module and the generic linked list to implement a program that sorts hex codes written in a file.

HW #6 compilation – Problem 1

One-step compilation for Problem 1: (this is what we instruct you to do)

```
==> gcc /share/ex_data/ex6/test/test_ex6.c hexCodeObject.c \
        /share/ex_data/ex6/hexCodeUtils.o \
        /share/ex_data/ex6/virtualHeap.o \
        -Wall -D TEST_1_3 \
        -o test_ex6_1_3
```

*We explain **-D TEST_1_3** next week*

- ✓ **test_ex6.c** – our test code (contains a main function)
- ✓ **hexCodeObject.c** – your source with implementation of HexCode object
- ✓ **hexCodeUtils.o** – pre-compiled object file for hex code utility functions
- ✓ **virtualHeap.o** – pre-compiled object file for virtual heap
- ✓ **test_ex6_1_3** – the final executable

- You can combine object files and source files in one command
- The compiler identifies object files and does not compile them
- The compiler compiles each source file and generates a temporary object file for it
- Finally, the compiler links all object files (pre-compiled and temporary)

HW #6 compilation – Problem 1

One-step compilation for Problem 1: (this is what we instruct you to do)

```
==> gcc /share/ex_data/ex6/test/test_ex6.c hexCodeObject.c \
        /share/ex_data/ex6/hexCodeUtils.o \
        /share/ex_data/ex6/virtualHeap.o \
        -Wall -D TEST_1_3 \
        -o test_ex6_1_3
```

*We explain **-D TEST_1_3** next week*

Incremental compilation for Problem 1: (manually do the different steps)

```
==> gcc -Wall -c hexCodeObject.c
==> gcc -Wall -c -D TEST_1_3 /share/ex_data/ex6/test/test_ex6.c \
              -o test_ex6_1_3.o

==> gcc hexCodeObject.o test_ex6_1_3.o \
        /share/ex_data/ex6/hexCodeUtils.o \
        /share/ex_data/ex6/virtualHeap.o \
        -o test_ex6_1_3
```

- First compilation creates object file **`hexCodeObject.o`**
- Second compilation creates object file **`test_ex6_1_3.o`**
- Third compilation links all object files into the program **`test_ex6_1_3`**

HW #6 compilation – Problem 2

One-step compilation for Problem 2: (this is what we instruct you to do)

Option 1

```
==> gcc -Wall /share/ex_data/ex6/virtualHeap.o        \
              /share/ex_data/ex6/genericLinkedList.o  \
              /share/ex_data/ex6/hexCodeUtils.o       \
              /share/ex_data/ex6/hexCodeObject.o      \
              hexCodeSort.c   -o hexCodeSort
```

- Uses our pre-compiled object file for the HexCode object

Option 2

```
==> gcc -Wall /share/ex_data/ex6/virtualHeap.o        \
              /share/ex_data/ex6/genericLinkedList.o  \
              /share/ex_data/ex6/hexCodeUtils.o       \
              hexCodeObject.c                         \
              hexCodeSort.c   -o hexCodeSort
```

- Uses your source file with an implementation of the HexCode object

HW #6 compilation – Problem 2

Incremental compilation for Problem 2: (this is what we instruct you to do)

Option 1

```
==> gcc -Wall -c hexCodeSort.c
==> gcc hexCodeSort.o \
              /share/ex_data/ex6/genericLinkedList.o \
              /share/ex_data/ex6/hexCodeUtils.o      \
              /share/ex_data/ex6/hexCodeObject.o     \
        -o hexCodeSort
```

- Uses our pre-compiled object file for the HexCode object

Option 2

```
==> gcc -Wall -c hexCodeSort.c
==> gcc -Wall -c hexCodeObject.c
==> gcc hexCodeObject.o hexCodeSort.o \
              /share/ex_data/ex6/genericLinkedList.o \
              /share/ex_data/ex6/hexCodeUtils.o      \
        -o hexCodeSort
```

- Uses your source file with an implementation of the HexCode object

➢ If you change your code in **`hexCodeSort.c`** and wish to recompile program, you do not have to compile **`hexCodeObject.c`** (second step in option 2), because it was not modified

➢ This can save precious time when building very large software packages